

KONGU ENGINEERING COLLEGE

SMART REFRIGERATOR MONITORING SYSTEM USING ESP32 AND DHT11 SENSOR

MECHATRONICS AND IOT

EXCERSICE – 08

HEMAVARSHINI S - 24MER017

KAVIN KUMARAN S - 24MER023

KOWSIKA K - 24MER026

VISHNUPRASATH B - 24MER062



KONGU ENGINEERING COLLEGE

— **TRANSFORM YOURSELF** —

SMART REFRIGERATOR MONITORING SYSTEM USING ESP32 AND DHT11/DHT22 SENSOR

1. Project Overview

Introduction

Conventional refrigerators do not continuously monitor and communicate their internal environmental conditions to the user. Improper temperature and humidity levels inside a refrigerator can affect food quality, increase spoilage, and lead to unnecessary energy consumption if left undetected.

The Smart Refrigerator Monitoring System is an ESP32-based IoT system that continuously monitors the internal temperature and humidity of a refrigerator using a DHT11/DHT22 sensor. The ESP32 processes the sensor readings, provides local alerts through a buzzer, and switches a relay to represent activation of a cooling/control system whenever the measured conditions move outside the desired range.

The measured parameters and system status are displayed locally through the Serial Monitor and are also uploaded to the Adafruit IO cloud platform over Wi-Fi, allowing the refrigerator condition to be monitored remotely through a cloud dashboard. The system does not use an OLED/LCD; all local readings are observed through the Serial Monitor.

The main objective is to demonstrate continuous, automated, condition-based monitoring and control of a refrigerator environment, supporting food-quality preservation and energy-efficient operation.

Objectives

- To continuously monitor refrigerator temperature.
- To monitor internal humidity conditions.
- To process sensor data using an ESP32.
- To provide an audible warning when abnormal conditions occur.
- To control a relay based on refrigerator conditions.
- To upload real-time sensor data to Adafruit IO.
- To monitor the refrigerator remotely through a cloud dashboard.
- To demonstrate energy-efficient, automatic monitoring and control.

2. Problem Statement

Conventional refrigerators do not continuously monitor and communicate their internal environmental conditions to the user. Improper temperature and humidity levels can affect food quality, increase spoilage, and lead to unnecessary energy consumption.

This can result in:

- Delayed detection of temperature or humidity deviations inside the refrigerator.
- Increased food spoilage due to undetected abnormal conditions.
- Unnecessary or inefficient operation of the cooling system.

- Lack of remote visibility into refrigerator conditions.
- Difficulty maintaining strict environmental limits for sensitive storage (e.g., pharmaceuticals, dairy).
- Dependence on manual, periodic checks, which is impractical for continuous monitoring.

Therefore, this project develops a Smart Refrigerator Monitoring System that continuously monitors temperature and humidity using a DHT11/DHT22 sensor, processes the readings using an ESP32, provides local alerts through a buzzer and relay, and uploads the measured parameters to Adafruit IO Cloud for remote monitoring.

3. Proposed Solution

The proposed system uses a DHT11/DHT22 sensor and a microcontroller to automate refrigerator condition monitoring. The DHT sensor, installed inside the refrigerator prototype, measures the internal temperature and relative humidity and sends the readings to the ESP32 through a single digital data pin.

The ESP32 processes these readings and compares them against predefined threshold values — a temperature range of 2–8°C and a humidity limit of below 70% RH — to determine the operating status.

When the measured parameters exceed the defined limits:

DHT Sensor → ESP32 → Status = ALERT → Buzzer ON, Relay

When the measured parameters remain within the defined limits:

DHT Sensor → ESP32 → Status = NORMAL → Buzzer OFF, Relay OFF

In parallel with the local alarm logic, the ESP32 continuously displays the readings and status on the Serial Monitor and connects to the internet through Wi-Fi to upload the sensor readings to Adafruit IO, enabling both immediate local response and remote monitoring of the refrigerator condition.

4. System Architecture

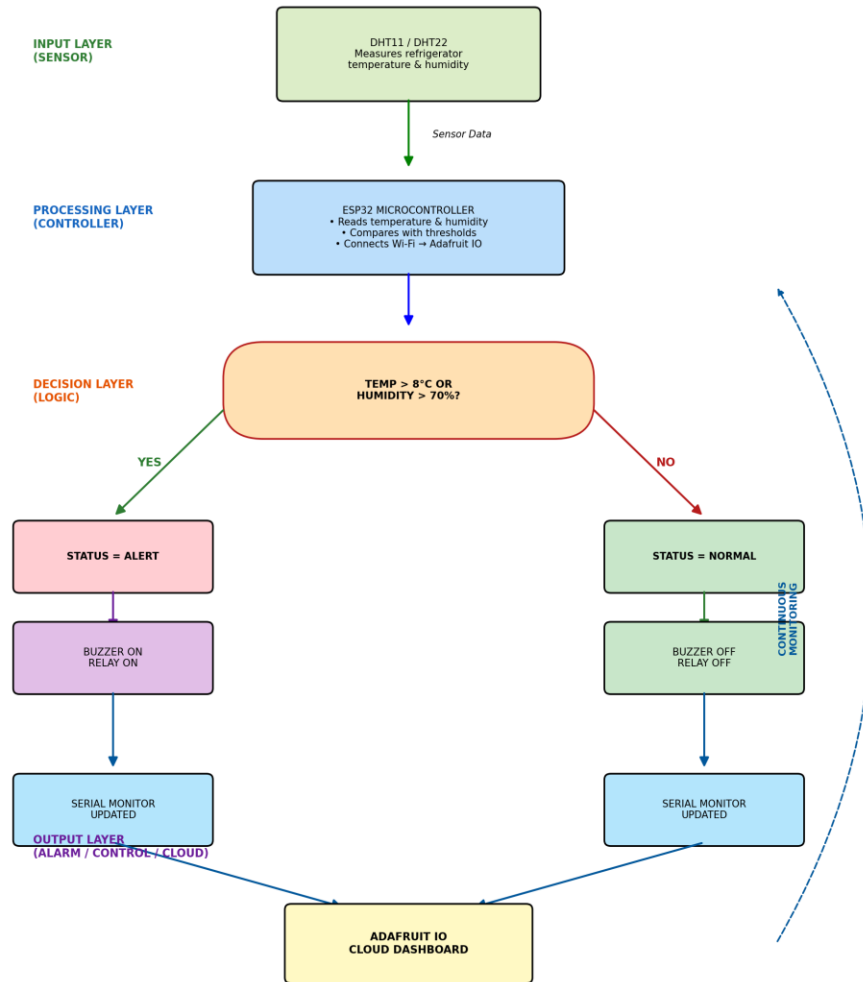
The overall architecture of the system is:

System Flow:

DHT11/DHT22 → ESP32 → Threshold Comparison → Buzzer + Relay + Serial Monitor → Adafruit IO Cloud

SMART REFRIGERATOR MONITORING SYSTEM

SYSTEM ARCHITECTURE FLOWCHART



5. Circuit Table (Hardware Connections)

DHT11 / DHT22

Sensor Pin	ESP32
VCC	3.3 V
DATA	GPIO 4
GND	GND

Buzzer

Buzzer	ESP32
VCC	3.3 V
GND	GND
Signal	GPIO 25

Relay

Relay	ESP32
VCC	5 V
GND	GND
IN	GPIO 26

The system does not use an OLED/LCD. All local readings are observed through the ESP32 Serial Monitor, while remote readings are displayed on Adafruit IO.

Power Supply

The ESP32 can be powered through USB or a regulated 5V supply. The DHT sensor and buzzer operate from the 3.3V rail, while the relay module is supplied from the ESP32's 5V pin. All grounds are tied to a common ESP32 GND.

6. Circuit Design

The circuit consists of three major sections:

6.1 ESP32 Microcontroller

The ESP32 is the main controller. It reads the DHT sensor data, compares it with the predefined thresholds, drives the buzzer and relay, and connects to Wi-Fi to upload data to Adafruit IO.

6.2 DHT11 / DHT22 Sensor

The DHT sensor is installed inside the refrigerator prototype and is connected to a single digital data pin.

DHT DATA → ESP32 GPIO 4

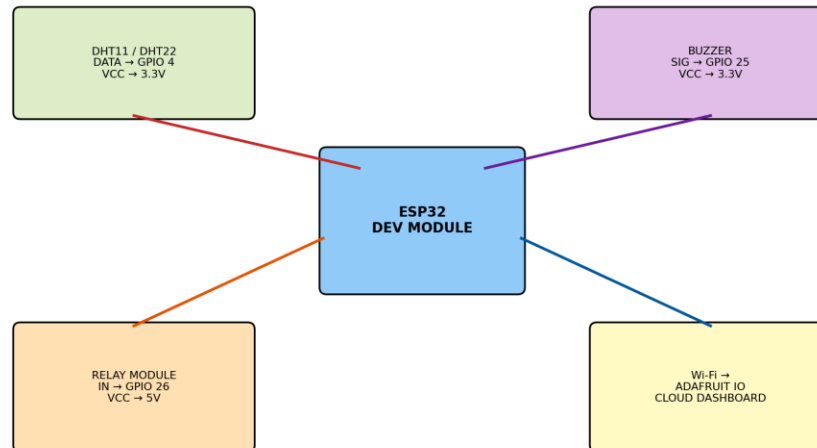
6.3 Buzzer and Relay

The buzzer provides an immediate audible alert during abnormal conditions, while the relay represents activation of the cooling/control system.

ESP32 GPIO 25 → Buzzer Signal | ESP32 GPIO 26 → Relay IN

CIRCUIT DESIGN:

REFRIGERATOR MONITORING - CIRCUIT CONNECTION DIAGRAM



7. Algorithm

1. Start the system.
2. Initialize the DHT sensor.
3. Initialize the buzzer and relay as outputs.
4. Connect the ESP32 to Wi-Fi.
5. Connect to Adafruit IO.
6. Read temperature from the DHT11/DHT22.
7. Read humidity from the DHT11/DHT22.
8. Display the readings on the Serial Monitor.
9. Compare the readings with the predefined thresholds ($\text{Temp} \leq 8^{\circ}\text{C}$, $\text{Humidity} \leq 70\%$).
10. If the readings are within limits, set Status = NORMAL, turn OFF the buzzer and relay.
11. If the readings exceed the limits, set Status = ALERT, turn ON the buzzer and relay.
12. Upload the temperature, humidity and status to Adafruit IO.
13. Repeat the process continuously.

8. Coding

The following Arduino program is designed for the ESP32 used in the project.

```
#include <WiFi.h>

#include "AdafruitIO_WiFi.h"

#include <DHT.h>

// =====

// WIFI DETAILS

// =====

#define WIFI_SSID    "IQOO Z10 5G"
#define WIFI_PASSWORD "gemcvdbr4vk8wbx"

// =====

// ADAFRUIT IO DETAILS

// =====

#define IO_USERNAME  "Vishnuprasath_B"
#define IO_KEY       "aio_iyDk23M0tgQZmIRTNevwApeuCmsQ"

// =====

// ADAFRUIT IO

// =====

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY,
                   WIFI_SSID, WIFI_PASSWORD);
```

```
// =====
```

```
// ADAFRUIT IO FEEDS
```

```
// =====
```

```
AdafruitIO_Feed *temperatureFeed = io.feed("temperature");
```

```
AdafruitIO_Feed *humidityFeed   = io.feed("humidity");
```

```
AdafruitIO_Feed *statusFeed     = io.feed("status");
```

```
// =====
```

```
// DHT SENSOR
```

```
// =====
```

```
#define DHT_PIN 4
```

```
#define DHT_TYPE DHT11
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```

```
// =====
```

```
// OUTPUTS
```

```
// =====
```

```
#define BUZZER_PIN 25
```

```
#define RELAY_PIN 26
```

```
// =====
```

```
// LIMITS
```

```
// =====
```



```
#define MAX_TEMP 8.0
#define MAX_HUM 70.0

// =====

// SETUP

// =====

void setup()
{
  Serial.begin(115200);

  // Start DHT
  dht.begin();

  // Output pins
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(RELAY_PIN, OUTPUT);

  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(RELAY_PIN, LOW);

  // -----

  // Connect to Adafruit IO

  // -----

  Serial.println();
```

```
Serial.println("=====");
```

```
Serial.println(" SMART REFRIGERATOR MONITOR");
```

```
Serial.println("=====");
```

```
Serial.println("Connecting to Adafruit IO...");
```

```
io.connect();
```

```
// Wait for connection
```

```
while (io.status() < AIO_CONNECTED)
```

```
{
```

```
    Serial.print(".");
```

```
    delay(500);
```

```
}
```

```
Serial.println();
```

```
Serial.println("Adafruit IO Connected!");
```

```
Serial.println("System Ready!");
```

```
Serial.println();
```

```
}
```

```
// =====
```

```
// LOOP
```

```
// =====
```

```
void loop()
```

```
{  
  // Keep Adafruit IO connection alive  
  io.run();  
  
  // -----  
  // Read DHT Sensor  
  // -----  
  
  float temperature = dht.readTemperature();  
  float humidity = dht.readHumidity();  
  
  // -----  
  // Sensor Error  
  // -----  
  
  if (isnan(temperature) || isnan(humidity))  
  {  
    Serial.println("ERROR: DHT SENSOR READING FAILED!");  
  
    digitalWrite(BUZZER_PIN, HIGH);  
  
    delay(2000);  
  
    return;  
  }  
  
  // -----
```

```
// Display readings on Serial Monitor
```

```
// -----
```

```
Serial.println("-----");
```

```
Serial.print("Temperature : ");
```

```
Serial.print(temperature);
```

```
Serial.println(" °C");
```

```
Serial.print("Humidity  : ");
```

```
Serial.print(humidity);
```

```
Serial.println(" %");
```

```
// -----
```

```
// Check refrigerator condition
```

```
// -----
```

```
if (temperature > MAX_TEMP || humidity > MAX_HUM)
```

```
{
```

```
// =====
```

```
// ALERT CONDITION
```

```
// =====
```

```
Serial.println("STATUS    : ALERT");
```

```
Serial.println("WARNING: Refrigerator condition abnormal!");
```

```
// Buzzer ON
digitalWrite(BUZZER_PIN, HIGH);

// Relay ON
digitalWrite(RELAY_PIN, HIGH);

// Send to Adafruit IO
temperatureFeed->save(temperature);
humidityFeed->save(humidity);
statusFeed->save("ALERT");
}

else
{
// =====

// NORMAL CONDITION

// =====

Serial.println("STATUS    : NORMAL");

// Buzzer OFF
digitalWrite(BUZZER_PIN, LOW);

// Relay OFF
digitalWrite(RELAY_PIN, LOW);

// Send to Adafruit IO
```

```
temperatureFeed->save(temperature);  
humidityFeed->save(humidity);  
statusFeed->save("NORMAL");  
}  
  
Serial.println("-----");  
Serial.println();  
  
// Wait before next reading  
delay(10000);  
}
```

Software Used

- Arduino IDE
- ESP32 Board Package
- DHT Sensor Library
- Adafruit Unified Sensor Library
- Adafruit IO Arduino Library
- Wi-Fi communication

Code Explanation

- GPIO 4 reads temperature and humidity from the DHT11/DHT22.
- GPIO 25 controls the buzzer.
- GPIO 26 controls the relay.
- tempLimit and humLimit define the abnormal-condition thresholds (8°C and 70% RH).
- AdafruitIO_Feed objects (temperature, humidity, status) upload the readings and status to the Adafruit IO dashboard.
- io.run() keeps the ESP32 connected to Adafruit IO and processes any pending cloud communication.

The threshold values should be adjusted according to the specific storage requirement — for example, stricter limits may be required for pharmaceutical or dairy storage.

9. Adafruit IO Cloud Monitoring

Three feeds were created in Adafruit IO:

1. Temperature Feed

Stores the temperature readings received from the ESP32.

2. Humidity Feed

Stores the humidity readings received from the DHT sensor.

3. Status Feed

Displays the operating condition:

- NORMAL
- ALERT

The Adafruit IO dashboard allows the refrigerator parameters to be monitored remotely through the internet.

10. Alert Mechanism

The system uses threshold-based decision-making.

Normal Condition

Temperature ≤ 8 °C

AND

Humidity $\leq 70\%$

↓

NORMAL

↓

Buzzer OFF

Relay OFF

Abnormal Condition

Temperature > 8 °C

OR

Humidity $> 70\%$

↓

ALERT

↓

Buzzer ON

Relay ON

The relay in the prototype represents the automatic cooling-control system. In a practical refrigerator, an appropriately rated and isolated control circuit would be required for the actual refrigeration system.

11. Serial Monitor Output

The ESP32 provides real-time readings through the Serial Monitor.

Example (normal condition):

```
-----  
Temperature : 5.60 °C  
Humidity   : 56.20 %  
STATUS     : NORMAL  
-----
```

During an abnormal condition:

```
-----  
Temperature : 10.30 °C  
Humidity    : 75.80 %  
STATUS      : ALERT  
WARNING: Refrigerator condition abnormal!  
-----
```

This allows the system to be tested without requiring a physical display.

12. Energy Efficiency

The system supports energy efficiency through automatic condition-based control. Instead of continuously operating a cooling-control output, the ESP32 makes a decision based on the measured environmental conditions.

When the monitored conditions are normal, the control output remains inactive. When abnormal conditions are detected, the relay is activated. This demonstrates the concept of smart and demand-based refrigeration control, which can potentially reduce unnecessary energy consumption.

13. Bill of Materials

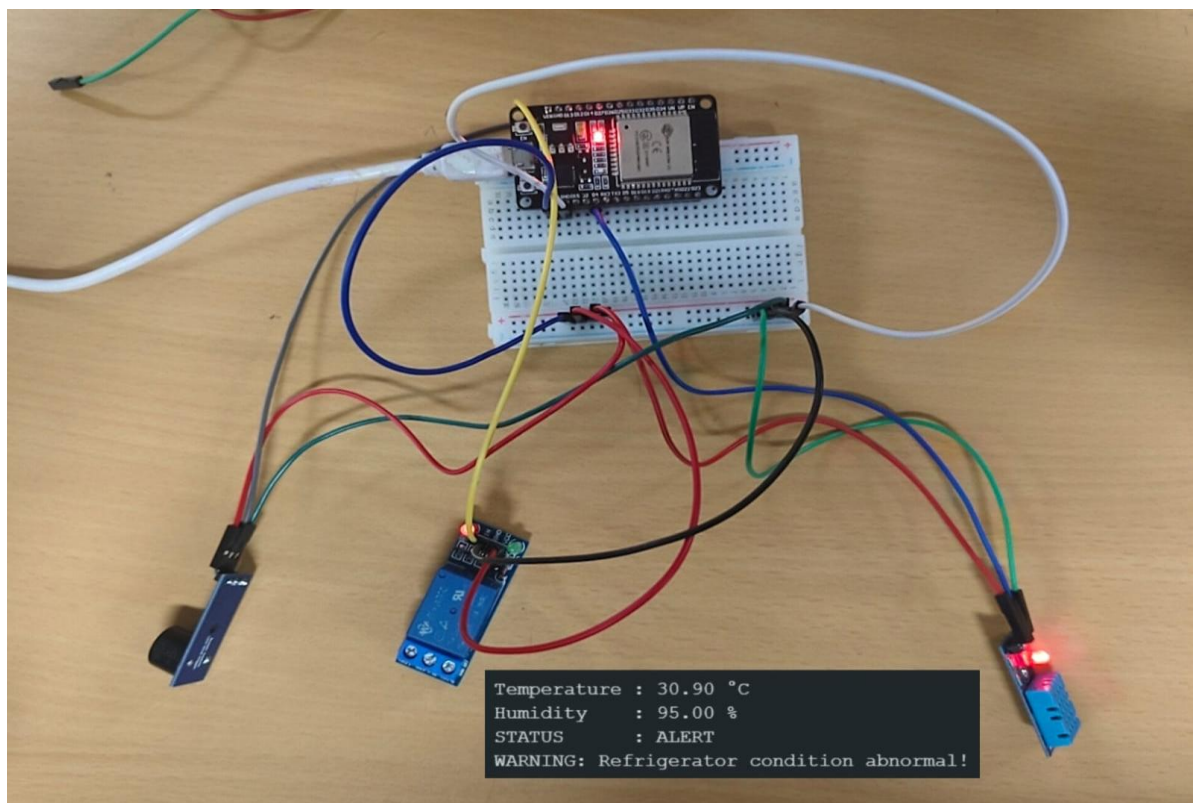
S.No.	Component	Quantity
1	ESP32 Development Board	1
2	DHT11 / DHT22 Sensor	1
3	Buzzer	1
4	Relay Module	1
5	Breadboard	1
6	Jumper Wires	Set
7	Wi-Fi Network	1
8	Adafruit IO	— (cloud platform)

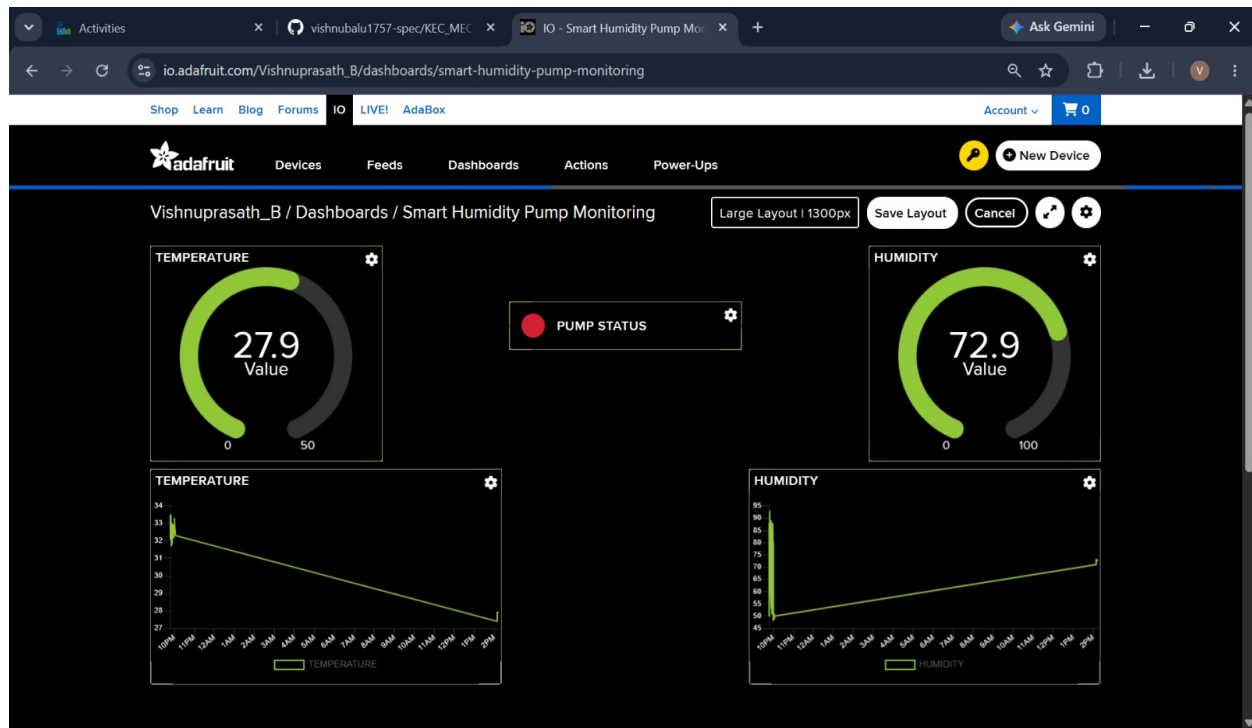
14. Prototype Implementation

The prototype was assembled using an ESP32 development board, a DHT11/DHT22 sensor, a buzzer, and a relay module, wired on a breadboard as per the circuit table above, and installed inside a refrigerator prototype.

The ESP32 was selected as the main controller because it provides sufficient GPIO pins for the sensor, buzzer and relay, along with built-in Wi-Fi connectivity for direct integration with Adafruit IO, without requiring an additional communication module. Since the system does not use an OLED/LCD, all local readings are observed through the Serial Monitor.

The prototype continuously reads temperature and humidity, compares them with the predefined thresholds, drives the buzzer and relay accordingly, and streams the readings to the Serial Monitor and the Adafruit IO dashboard.





15. Results

The developed prototype successfully demonstrates:

1. Temperature measurement using DHT11/DHT22.
2. Humidity measurement using DHT11/DHT22.
3. Real-time data processing using ESP32.
4. Local monitoring through the Serial Monitor.
5. Abnormal-condition detection using predefined thresholds.
6. Automatic buzzer activation during alert conditions.
7. Relay activation as a simulated cooling-control output.
8. Cloud data transmission through Wi-Fi.
9. Remote monitoring using the Adafruit IO dashboard.

Condition	Sensor Condition	Buzzer	Relay	Status
Normal storage condition	Temp $\leq 8^{\circ}\text{C}$, Humidity $\leq 70\%$	OFF	OFF	NORMAL
Abnormal storage condition	Temp $> 8^{\circ}\text{C}$ or Humidity $> 70\%$	ON	ON	ALERT

16. Advantages

- Real-time temperature monitoring.
- Humidity monitoring.
- Automatic abnormal-condition detection.
- Audible warning using a buzzer.

- Relay-based automatic control.
- Cloud-based remote monitoring.
- Low-cost prototype.
- Wi-Fi connectivity using ESP32.
- Data can be viewed through Adafruit IO.
- Suitable for IoT-based refrigeration applications.

17. Applications

The developed concept can be adapted for:

- Domestic refrigerators.
- Food storage units.
- Cold storage facilities.
- Pharmaceutical storage.
- Dairy product storage.
- Laboratory refrigerators.
- Temperature-sensitive material storage.
- IoT-based food preservation systems.

18. Limitations

- DHT11 has limited temperature accuracy and response time.
- DHT11 is not ideal for precise refrigerator temperature measurement.
- The prototype does not directly control an actual refrigerator compressor.
- Wi-Fi connectivity is required for cloud monitoring.
- The relay control is demonstrated as a prototype output.

For a more advanced version, a DS18B20 or other dedicated temperature sensor can be used for more reliable low-temperature measurement.

19. Conclusion

The Smart Refrigerator Monitoring System using ESP32 and DHT11/DHT22 successfully demonstrates an IoT-based approach for monitoring refrigerator environmental conditions. The system measures temperature and humidity continuously and processes the data using the ESP32.

When abnormal conditions are detected, the system activates a buzzer and relay to provide an immediate response. The measured parameters and system status are also transmitted to Adafruit IO, enabling remote monitoring through a cloud dashboard.

Thus, the project provides a low-cost, connected, and expandable solution for smart food-preservation monitoring, while demonstrating the integration of sensors, embedded control, wireless communication, cloud monitoring, and automatic alerting.